

TeamUp

Managerial Round · Interview Preparation Document

Full-stack project · React 19 + Vite + Tailwind 4 · Spring Boot 3.3 + JPA + JWT + H2

Explain · Defend · Discuss

Sections 1-11: project mastery · Section 12: 119 interview Q&A · Section 13: cheat sheet

Table of Contents

Tip: in Word, right-click the table below and choose "Update Field" to refresh page numbers.

Table of Contents	2
1. Project in 60 Seconds	5
Elevator pitch (15 seconds)	5
2-minute "Tell me about your project" speech (ready to speak)	5
2. Problem and Solution	5
Problem statement	5
Target users	5
What makes it useful	6
Key features (short list)	6
3. Tech Stack and "Why this?"	7
Backend	7
Frontend	7
4. Architecture and Design	8
Overall shape: two-tier, layered monolith	8
Package structure (backend) — package by feature, not by layer	8
Frontend structure	8
Design patterns and principles (with real locations)	8
5. How It Works: End-to-End Flows	10
Flow A — Login (authentication)	10
Flow B — Create a post (main entity create)	10
Flow C — Join request + accept (the core business flow)	10
Flow D — Read + filter the feed	11
Flow E — Update my profile	11
Frontend-to-backend conventions (one line each)	11
6. Database Design	12
Entities and tables	12
Relationships	12
Why this schema (design reasoning)	12
Keys, indexes, normalization	13
7. Code Deep Dive	14
7.1 JWT creation — JwtService.generateToken()	14
7.2 The auth filter — JwtAuthFilter.doFilterInternal()	14
7.3 Security rules — SecurityConfig.securityFilterChain()	14
7.4 Partial update (PATCH) — UserController.applyUpdate()	15

7.5 Error normalization — ApiExceptionHandler	15
7.6 Frontend HTTP core — api.js request()	15
7.7 Annotations cheat-sheet (name + one-line meaning).....	16
8. How It Was Built	17
Git history insight	17
Development approach (as the history shows).....	17
Challenges faced and how they were solved (from the code)	17
9. Security, Performance and Error Handling.....	18
Security checklist (what the repo actually does).....	18
Error handling strategy.....	18
Performance considerations (honest state).....	18
10. Testing, Deployment and DevOps	19
Testing that exists.....	19
How it is run (the "deployment" story)	19
CI/CD.....	19
11. Limitations, Improvements and Future Scope	20
Honest weaknesses in the current code	20
How I would scale to 1 lakh (100k) / 1 million users	20
If I rebuilt it from scratch.....	21
12. Interview Q&A Bank	21
a) Project overview and motivation	21
b) "Why" questions (stack, DB, architecture, choices).....	24
c) "How" questions (flows and behavior).....	27
d) Code-level questions (specific classes in this repo)	31
e) Database questions (schema, joins, transactions, indexes).....	35
f) API and backend design questions.....	38
g) Frontend questions	41
h) Security questions	44
i) Scalability and performance questions	47
j) Challenges, bugs, failures and learnings	50
k) Managerial and behavioral questions (tied to this project)	52
l) Tricky / cross questions (and handling "I don't know").....	55
m) Scenario questions ("what if..." under pressure)	58
13. Cheat Sheet (one-page rapid revision)	60
Stack in one line	60
Key numbers (memorize)	60
Three flows to be able to draw from memory	60

Top 15 questions to prepare cold 60

Top 10 things NEVER to say..... 61

1. Project in 60 Seconds

Elevator pitch (15 seconds)

TeamUp is a full-stack web platform where students and early-career builders **post project ideas** (hackathons, open source, class projects, startup ideas), **discover teammates** by skills and availability, **apply to open roles**, and **run the whole team-formation loop** — from posting to accept/reject to notifications — in one place.

2-minute "Tell me about your project" speech (ready to speak)

INTERVIEW TIP: Speak it out loud twice before the interview. Aim for ~90-120 seconds. Pause at the arrows — they are your natural breath points.

"I built **TeamUp**, a full-stack team-formation platform for students. The problem it solves is simple: students want to join hackathons or open-source projects, but finding teammates is chaotic — WhatsApp forwards and word of mouth, with no idea who is actually free or what skills they bring.

On the **frontend** I used **React 19 with Vite** and **Tailwind CSS 4** — a single-page app with 14 routes: feed, post detail, dashboard, requests, notifications, profiles, and auth screens. State is simple: a small ``api.js`` fetch wrapper, a ``useFetch`` hook, and React Router for navigation.

On the **backend** I built a **Spring Boot 3.3 REST API on Java 22** with **Spring Security + JWT**, **Spring Data JPA**, and an **H2 in-memory database** so anyone can run it with zero setup. It is a layered monolith: controllers handle HTTP, repositories handle SQL, and a single ``@RestControllerAdvice`` class normalizes every error into ``{status, message}``.

The core flow is: a user signs up, publishes a post with roles and required skills, others send join requests, the creator accepts or rejects, and both sides get notifications. Everything is protected by a **stateless JWT filter** — ``JwtAuthFilter`` reads the ``Authorization: Bearer`` header, validates the token, and sets the Spring Security context.

I wrote **MockMvc integration tests** for the risky paths: login success and failure, a non-creator trying to edit someone else's post (must get 403), and the full join-request loop including the duplicate-request 409 case. The repo has **13 clean commits** and a README with setup, routes, and API docs."

REMEMBER: End with: "The parts I would strengthen next are pagination and a proper service layer — I know exactly where the current design bends." Managers love candidates who already know their weak spots.

2. Problem and Solution

Problem statement

- Students form teams informally — chat groups, classroom announcements, friends-of-friends.
- You cannot see **who is open to work, what skills they have, or when they are free**.
- Project creators advertise roles in chat messages that scroll away; there is no status tracking (pending / accepted / rejected).
- After formation, there is no place to see "my projects, my requests, my notifications" in one view.

Target users

- **Primary:** college students joining hackathons, class projects, and open-source programs.
- **Secondary:** student club leads and startup founders looking for early collaborators.

What makes it useful

- Structured **posts** with category, roles, skills, deadline, and online/offline/hybrid mode.
- **Skill-based recommendations** — `PostController.recommended()` scores posts by overlap between the user skills and each role skill list.
- A **request lifecycle** with real states and duplicate protection, not chat messages.
- Runs locally with **zero external dependencies** — H2 in-memory DB + demo seed data.

Key features (short list)

- Auth: signup, login (JWT), password-recovery screen.
- Posts: create, browse, filter by category/mode/status, search by title/description, edit, delete (creator only).
- Join requests: submit, list, accept/reject with notifications.
- Comments on posts.
- Dashboard aggregating my posts, sent requests, received requests, notifications.
- Profiles: skills, interests, links, timeline, availability — fully editable.
- People directory with year/availability/skill filters; saved posts stored in the browser.

3. Tech Stack and "Why this?"

Every choice below is in the repo — ``pom.xml``, ``package.json``, ``vite.config.js``, ``application.yml``.

Backend

Technology	Why I chose it	Alternative I considered
Java 22 + Spring Boot 3.3.5	Industry standard for Java full-stack roles; auto-configured server, DI, and testing in one framework	Plain Java servlets / Jakarta EE — too much manual wiring
Spring Security + JWT	Declarative request rules + stateless JWT fits a REST API	Session cookies — ties server to sessions, harder to scale horizontally
Spring Data JPA (Hibernate)	Repository interface = no boilerplate SQL; DB can be swapped	JDBC / MyBatis — write every query by hand
H2 in-memory DB	Zero setup for demos and tests; <code>`ddl-auto: create-drop`</code> auto-creates schema	MySQL/Postgres — production-ready but needs install/config for a demo
Bean Validation (@Valid)	Declarative input checks on DTO records	Manual if-checks everywhere (I still use some — see Section 11)
SLF4J logging + @RestControllerAdvice	One place converts every exception to a clean JSON body	Returning ResponseEntity error objects from every method

Frontend

Technology	Why I chose it	Alternative I considered
Vite 7 + React 19	Instant dev server, native ESM, tiny config; React is what I wanted to deep-dive	Next.js — SSR/SEO not needed; auth-gated app is a pure SPA
Tailwind CSS 4 (@tailwindcss/vite)	Utility classes in JSX, design tokens in CSS via <code>@theme</code> ; no CSS-file sprawl	Styled components — extra runtime; plain CSS — hard to keep consistent
react-router-dom 7	Client-side routing for all 14 screens	React context-only navigation — no URL deep links
lucide-react	Tree-shaken inline SVG icons	Icon font — extra requests, blurry at some sizes
Custom api.js + useFetch	One fetch wrapper (base URL, JSON, Bearer token, errors) and one data hook	Axios + Redux Toolkit — overkill for this app size
pnpm	Fast, strict node_modules; native Vite integration	npm/yarn — slower installs

INTERVIEW TIP: Never say "because it was popular." Always tie the choice to a constraint: "stateless API + horizontal scaling → JWT", "no SEO on auth screens → SPA instead of Next.js".

4. Architecture and Design

Overall shape: two-tier, layered monolith

Frontend **SPA** talks JSON over HTTP to a **Spring Boot monolith**. Inside the monolith: **Controller -> Repository -> DB** (no separate service layer — a deliberate simplification, and one of my named improvements).

```
Browser (React SPA, port 5173)
  | fetch() + JSON + "Authorization: Bearer <jwt>"
  v
Spring Boot (port 4000)
  SecurityFilterChain --> JwtAuthFilter (validate token, set SecurityContext)
  v
@RestController (PostController / AuthController / ...)
  v
JpaRepository (PostRepository / UserRepository / ...)
  v
H2 in-memory DB (users, posts, join_requests, comments, notifications)

Errors anywhere --> ApiExceptionHandler --> { "status": 4xx/5xx, "message": "..." }
```

Package structure (backend) — package by feature, not by layer

- **com.teamup.controller** — 7 HTTP entry points (`AuthController`, `PostController`, `UserController`, `CommentController`, `JoinRequestController`, `MeController`, `HealthController`).
- **com.teamup.user / post / comment / request** — each feature package holds its **entity + repository** (and DTOs like `PostRequest`, `UserView`).
- **com.teamup.security** — `JwtService` (create/verify tokens) + `JwtAuthFilter` (per-request auth).
- **com.teamup.config** — `SecurityConfig` (rules, CORS, bcrypt) + `DataSeeder` (demo data).
- **com.teamup.common** — `ApiExceptionHandler` (global error translation).

Frontend structure

- **src/lib** — `api.js` (HTTP client + view models), `use-fetch.js` (data hook), `saved.js` (localStorage saved posts).
- **src/components/platform-shell.jsx** — shared shell: `PlatformShell`, `Avatar`, `SectionCard`, `PageButton`.
- **src/pages/** — one file per screen (14 routes wired in `App.jsx`).
- **src/index.css** — Tailwind v4 `@theme` tokens (ink, paper, orange...) + `blueprint` grid utilities.

Design patterns and principles (with real locations)

- **Repository pattern** — `JpaRepository<T, Long>` interfaces; controllers never write SQL.
- **DTO / View records** — `PostView.from(post)`, `UserView.from(user)` flatten entities so lazy/EAGER internals never leak to the API.
- **Front Controller** — Spring `DispatcherServlet` routes every URL to a controller method.
- **Filter chain (interceptor style)** — `JwtAuthFilter` extends `OncePerRequestFilter` runs once per request before controllers.
- **Cross-cutting advice** — `@RestControllerAdvice ApiExceptionHandler` centralizes error responses (single responsibility).

- **Template-ish partial update** — ``applyUpdate()`` / ``applyPostFields()`` copy only non-null fields (PATCH semantics).
- **DRY on frontend** — every screen shares ``useFetch`` and one ``request()`` function instead of bespoke fetch code.

NOTE: There is **no service layer** in this repo — business rules live in controllers. I say this openly in interviews and explain I would extract a ``PostService`` if the logic grew.

5. How It Works: End-to-End Flows

INTERVIEW TIP: Pick ONE flow and memorize it deeply (I recommend join-request). Being deep on one beats being shallow on five.

Flow A — Login (authentication)

```
LoginPage.handleSubmit(e)                src/pages/login.jsx
-> api.login(email, password)             src/lib/api.js
-> POST /api/auth/login {email, password}
-> SecurityConfig: "/api/auth/**" is permitAll (no token needed yet)
-> AuthController.login()
    userRepository.findByEmail(lowercased email)
    passwordEncoder.matches(raw, hashed) -> false => 401 "Invalid email or password"
    jwtService.generateToken(id, role, email) -> signed JWT (15 min)
<- 200 { user: UserView, accessToken: "eyJ..." }
-> setToken(accessToken) saves in localStorage key "teamup_token"
-> navigate("/posts")
```

Every later request:

```
api.request() attaches Authorization: Bearer <token>
JwtAuthFilter validates signature+expiry -> sets SecurityContext
controller reads principal (Long userId) via Authentication
```

Flow B — Create a post (main entity create)

```
create-post page -> api.createPost({...})
-> POST /api/posts (+ Bearer token)
-> SecurityConfig: anyRequest().authenticated() -> 401 if token missing/invalid
-> PostController.create(authentication, request)
    currentUserId(authentication) -> 401 if no principal
    manual check: title/description/category required -> else 400
    new Post(); applyPostFields() builds PostRole list + currentMembers
    postRepository.save(post)
        Hibernate @PrePersist sets createdAt/updatedAt
        cascade = ALL persists post_roles rows too
<- 201 PostView (JSON)
-> UI shows the new post in the feed (refetch via useFetch)
```

Flow C — Join request + accept (the core business flow)

```
1. Requester: post-detail page -> api.submitJoinRequest(id, {role, message})
-> POST /api/posts/{id}/join
-> JoinRequestController.submit()
    guard: requester == creator -> 400 "cannot request to join your own post"
    guard: pending dup exists -> 409 CONFLICT
    save JoinRequest(status=pending)
    notify(post.creator, Type.joinRequest, "<name> requested to join <title>")
<- 201 JoinRequestView

2. Creator: requests page -> api.getPostRequests(id) [owner check: 403 if not creator]

3. Creator: api.respondToRequest(postId, requestId, "accept")
-> PATCH /api/posts/{id}/requests/{requestId} {action: "accept"}
```

```

-> JoinRequestController.respond()
  owner check (currentOwnerPost) -> 403 if someone else
  status must be pending -> 409 if already handled
  accept: request.status = accepted; addMember(post, request); postRepository.save(post)
        notify(requester, Type.requestAccepted, "You joined <title>")
  reject: request.status = rejected + notify(Type.requestRejected)
<- 200 JoinRequestView

```

4. Requester opens /notifications -> unread badge via GET /api/me/notifications

Flow D — Read + filter the feed

```

posts.jsx: useFetch(() => api.getPosts(), [])
-> GET /api/posts
-> PostController.list(): postRepository.findAllOrderByCreatedAtDesc()
  optional server-side filters: category / mode / status / search (in-memory .filter)
<- [PostView]
-> React state + useMemo filters again in the browser (instant, no round trip)
Result: server gives newest-first list; the page narrows it locally as you type.

```

Flow E — Update my profile

```

profile-edit.jsx loads: api.getMe() -> GET /api/users/me (auth required)
user edits form -> api.updateMe({skills, bio, links, ...})
-> PATCH /api/users/me
-> UserController.updateMe(): applyUpdate(user, request) – only non-null fields applied
  availability enum parsed defensively (bad value silently ignored)
-> userRepository.save(user) -> @PreUpdate refreshes updatedAt
<- 200 UserView

```

Frontend-to-backend conventions (one line each)

- **Base URL:** `VITE\_API\_URL` env, default `http://localhost:4000` (`src/lib/api.js`).
- **Auth:** header `Authorization: Bearer <jwt>`; token persisted in **localStorage** (`teamup\_token`).
- **Errors:** non-2xx -> `ApiError` with server `message`; pages render it inline (e.g., login red error text).
- **Data fetching:** `useFetch(fn, deps)` returns `{data, loading, error}` and cancels stale updates with an `active` flag.

6. Database Design

NOTE: Schema is generated by Hibernate (`ddl-auto: create-drop`) — there are **no SQL migration files in the repo**. The table shapes below come from the `@Entity` classes.

Entities and tables

Entity (file)	Table	Key columns
User (user/User.java)	users	id PK, name, email, password (bcrypt), college, college_year, bio, location, availability (enum str), role, verified, created_at, updated_at
Post (post/Post.java)	posts	id PK, title, description, category (enum str), creator_id FK->users, status, mode, deadline, event_link, external_link, timestamps
PostRole (post/PostRole.java)	post_roles	id PK, post_id FK->posts, role_name, count, + skills element-collection table
JoinRequest (request/JoinRequest.java)	join_requests	id PK, post_id FK, requester_id FK, role, message, status (pending/accepted/rejected), responded_at, timestamps
Comment (comment/Comment.java)	comments	id PK, post_id FK, author_id FK, body, timestamps
Notification (request/Notification.java)	notifications	id PK, user_id FK, type (enum str), message, related_post_id FK, related_request_id FK, read, timestamps

Relationships

- **Post N-1 User** — `@ManyToOne(fetch = EAGER)` `creator`; a user has many posts.
- **Post 1-N PostRole** — `@OneToMany(mappedBy="post", cascade=ALL, orphanRemoval=true)`; deleting a post deletes its roles, replacing the list removes stale rows.
- **Post N-1 JoinRequest, User N-1 JoinRequest** — who applied, to which post.
- **Post N-1 Comment, User N-1 Comment** — comment thread per post.
- **Notification** — N-1 to user, and optional N-1 to post + request (what it is about).
- **Value-object style:** `User.skills`, `User.interests`, `User.timeline`, `Post.currentMembers` are `@ElementCollection` — stored in separate child tables keyed to the owner, no independent identity.

Why this schema (design reasoning)

- **Enums stored as strings** (`@Enumerated(EnumType.STRING)`) — readable rows, survives enum reordering; trade-off: slightly larger storage.
- **`currentMembers` is a snapshot** (userId + userName copied) — display survives a user renaming themselves; it is not a FK join.
- **`rolesRequired` is a real child entity** — it has an id and is cascade-managed, because posts update their role lists over time (orphanRemoval cleans up).
- **Timestamps via `@PrePersist` / `@PreUpdate`** on every entity — DB rows always carry created/updated without controller effort.

Keys, indexes, normalization

- **PKs:** `GenerationType.IDENTITY`` (auto-increment) on every entity.
- **FKs:** declared with `@JoinColumn`` (`creator_id``, `post_id``, ...).
- **Indexes:** **none declared in the repo** (no `@Index`` annotations, no migration DDL). H2 auto-indexes PKs only. This is a known gap — see Section 11.
- **Unique constraint on email: not declared** — only an application-level `existsByEmail()`` check in `AuthController.signup()`` (race-prone; I would add `@Column(unique = true)``).
- **Normalization:** skills/interests/timeline moved to child tables (3NF-ish); member snapshot and embedded `UserLinks`` are deliberate denormalizations for simplicity.
- **Query style:** derived query methods only, e.g. `findByUserIdOrderByCreatedAtDesc(Long)`` — Spring Data parses the method name into SQL. No `@Query`` JPQL in the repo.

7. Code Deep Dive

7.1 JWT creation — JwtService.generateToken()

backend/.../security/JwtService.java

```
public String generateToken(Long userId, String role, String email) {
    Date now = new Date();
    Date expiry = new Date(now.getTime() + expirationMillis); // default 900000 ms = 15 min
    return Jwts.builder()
        .subject(String.valueOf(userId)) // "who" = user id (sub claim)
        .claims(Map.of("role", role, "email", email)) // extra data in the token
        .issuedAt(now)
        .expiration(expiry)
        .signWith(key) // HMAC-SHA key from app.jwt.secret
        .compact();
}
```

Line by line: it builds a **signed, expiring JSON token**. `subject` = the user id; `role`/`email` ride along as claims so the filter does not hit the DB on every request. `signWith(key)` proves nobody tampered with it — `JwtAuthFilter` later calls `parseSignedClaims()` which throws on bad signature or expiry, and `isValid()` swallows that into `false`.

7.2 The auth filter — JwtAuthFilter.doFilterInternal()

backend/.../security/JwtAuthFilter.java (skeleton)

```
String token = extractToken(request); // "Authorization: Bearer x" or "accessToken" cookie
if (token != null && SecurityContextHolder...
    ...getContext().getAuthentication() == null) {
    if (jwtService.isValid(token)) { // signature + expiry check
        Long userId = Long.valueOf(jwtService.extractUserId(token));
        var authToken = new UsernamePasswordAuthenticationToken(
            userId, null, List.of(new SimpleGrantedAuthority("ROLE_" + role)));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
filterChain.doFilter(request, response); // ALWAYS continue the chain
```

Two interview gold points: **(1)** it never blocks the chain when the token is missing — it just leaves the context empty, so `permitAll` endpoints still work while protected endpoints later fail with 401; **(2)** the principal is a plain `Long userId`, which is why controllers write `authentication.getPrincipal().instanceof Long id`.

7.3 Security rules — SecurityConfig.securityFilterChain()

backend/.../config/SecurityConfig.java (trimmed)

```
.csrf(...disable) // stateless API, no cookie sessions to protect
.sessionManagement(... STATELESS) // server remembers nothing between requests
.cors(... corsConfigurationSource()) // localhost:3000 allowed (see Section 11!),
// methods GET POST PATCH PUT DELETE OPTIONS
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/health", "/api/auth/**", "/h2-console/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/posts/**", "/api/users/**").permitAll()
    .anyRequest().authenticated() // everything else needs a valid JWT
.addFilterBefore(jwtAuthFilter,
    UsernamePasswordAuthenticationFilter.class);
```

Read this table as a sentence: "reads are public; auth endpoints are public; everything else needs a token."

Passwords: `new BCryptPasswordEncoder(12)` — strength 12 means $2^{12} = 4096$ bcrypt rounds.

7.4 Partial update (PATCH) — UserController.applyUpdate()

backend/.../controller/UserController.java (pattern)

```
if (request.name() != null) user.setName(request.name()); // only send fields to change
if (request.links() != null) {                             // merge nested object,
    User.UserLinks links = user.getLinks() == null
        ? new User.UserLinks() : user.getLinks();         // create if absent,
    if (request.links().github() != null)                  // then override
        links.setGithub(request.links().github());
    user.setLinks(links);
}
```

This is how I implement **PATCH semantics**: null means "do not touch". It avoids the classic bug where a partial form save wipes other fields. The same helper pattern is `PostController.applyPostFields()`.

7.5 Error normalization — ApiExceptionHandler

backend/.../common/ApiExceptionHandler.java

```
@RestControllerAdvice // one class, applies to ALL controllers
public class ApiExceptionHandler {

    @ExceptionHandler(ResponseStatusException.class) // 404/409/403 thrown by controllers
    public ResponseEntity<Map<String, Object>> handleResponseStatus(...) { ... }

    @ExceptionHandler(MethodArgumentNotValidException.class) // @Valid failures -> 400

    @ExceptionHandler(Exception.class) // catch-all -> logs + 500
    public ResponseEntity<...> handleGeneric(Exception ex) {
        log.error("Unhandled exception", ex); // server logs full stack trace,
        return build(500, "Internal server error"); // client sees safe message
    }
}
```

Client always receives the same shape: `{"status": 409, "message": "Email already registered"}` — so `api.js` can show `error.message` uniformly on every page.

7.6 Frontend HTTP core — api.js request()

src/lib/api.js (trimmed)

```
async function request(path, options = {}) {
    const headers = { "Content-Type": "application/json", ...options.headers };
    const token = getToken(); // localStorage "teamup_token"
    if (token) headers.Authorization = `Bearer ${token}`;

    const response = await fetch(API_URL + path, { ...options, headers });

    if (!response.ok) { // 4xx / 5xx
        const data = await response.json().catch(() => ({}));
        throw new ApiError(response.status, data.message || "Request failed");
    }
    return response.status === 204 ? undefined : response.json();
}
```

Every screen calls thin wrappers (``api.getPosts()``, ``api.login()``) — one place to change base URL, auth header, or error handling. Companion hook ``useFetch`` in ``src/lib/use-fetch.js`` manages ``loading/error`` state and ignores results from unmounted components via the ``active`` flag.

7.7 Annotations cheat-sheet (name + one-line meaning)

Annotation	What it does
<code>@RestController</code>	Class handles HTTP + returns JSON (not a view).
<code>@RequestMapping("/api/posts")</code>	Base URL for every method in the class.
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PatchMapping</code> / <code>@DeleteMapping</code>	Bind HTTP verb + path to a method.
<code>@PathVariable</code> / <code>@RequestParam</code> / <code>@RequestBody</code>	Extract from URL segment / query string / JSON body.
<code>@Valid</code> + <code>@NotBlank</code> / <code>@Email</code> / <code>@Size</code>	Spring validates the record before the method runs -> 400 on failure.
<code>@Entity</code> / <code>@Table(name="posts")</code>	Class maps to a DB table.
<code>@Id</code> + <code>@GeneratedValue(IDENTITY)</code>	Primary key, auto-increment.
<code>@ManyToOne</code> / <code>@OneToMany</code>	Many rows point to one parent / parent owns children.
<code>@ElementCollection</code>	List of simple values or embeddables in a side table.
<code>@Enumerated(EnumType.STRING)</code>	Store enum as its name ("Open"), not ordinal (0).
<code>@PrePersist</code> / <code>@PreUpdate</code>	Lifecycle hooks -> set <code>createdAt</code> / <code>updatedAt</code> automatically.
<code>@Configuration</code> + <code>@Bean</code>	Declare objects (PasswordEncoder, filter chain) for the container.
<code>@EnableWebSecurity</code>	Turn on Spring Security auto-configuration for this app.
<code>@Service</code> / <code>@Component</code>	Mark a class as a managed bean (JwtService is a <code>@Service</code>).
<code>@Transactional</code>	Method runs in one DB transaction (used on <code>DataSeeder.run</code>).
<code>@RestControllerAdvice</code>	Global exception handler applied to all controllers.
<code>@Value("\${app.jwt.secret}")</code>	Inject config value from <code>application.yml</code> / env var.

8. How It Was Built

NOTE: Everything here is inferred from **git history + README + code**. Anything the repo cannot prove is marked "Not found in repo".

Git history insight

- **13 commits, all on one day** (2026-10-02), clean conventional messages, no fix-up noise.
- Build order visible in history: **scaffold -> design tokens -> entry+routes -> API client -> shell -> pages one feature at a time -> backend -> README**.
- Each feature area got its own commit (e.g. "feat: add posts feed and post detail pages") — reviewable, revertable units.
- **Not found in repo:** an earlier Next.js version of this project is referenced in conversations but predates `git init`, so the history starts at the Vite scaffold. If asked, say the project was **re-architected from a pages-framework prototype to a Vite SPA** and the git history begins at that rewrite.

Development approach (as the history shows)

1. **Contract first** — the frontend `api.js` view models (`PostView`, `UserView`...) were defined to match the JSON the backend would return; both sides kept the same field names.
2. **Vertical walking skeleton** — scaffold + one route + health endpoint before features.
3. **Shared shell** — `PlatformShell` built once so every page inherits sidebar/topbar for free.
4. **Feature-page sprints** — feed, dashboard, requests, notifications, auth, profile each as a separate commit.
5. **Backend last on purpose** — UI could be developed against a known contract; H2 + `DataSeeder` made the API instantly demo-able.
6. **Tests for the risky paths** — auth failure, authorization failure, and the join-request state machine (see Section 10).

Challenges faced and how they were solved (from the code)

- **Stateless auth across two apps** — solved with one shared `request()` wrapper that injects the Bearer token, so no page forgets it.
- **Partial updates without wiping data** — solved with `applyUpdate()` null-checks; nested `links` are merged, not replaced blindly.
- **Duplicate join requests** — application-level guard in `JoinRequestController.submit()` returns 409; frontend surfaces the server message.
- **Error consistency across ~23 endpoints** — solved once in `ApiExceptionHandler` instead of per-method try/catch.
- **Demo usability** — `DataSeeder` (a `CommandLineRunner`) seeds 5 users + 4 posts with one shared demo password so the app is demo-able in 30 seconds.

COMMON MISTAKE: Do not claim a CI pipeline, Docker setup, or staging environment — none exist in the repo. If asked, answer: "Not deployed yet; locally I run `mvnw spring-boot:run` + `pnpm dev`. My next step would be GitHub Actions + Docker."

9. Security, Performance and Error Handling

Security checklist (what the repo actually does)

- **Passwords:** BCrypt with strength **12** (``SecurityConfig.passwordEncoder()``); only the hash is stored; login uses ``passwordEncoder.matches()``.
- **Login error hygiene:** same message "Invalid email or password" for unknown email and wrong password — no user enumeration.
- **JWT:** HS256 signed, **15-minute expiry** (``app.jwt.expiration-ms: 900000``), secret overridable via ``APP_JWT_SECRET`` env.
- **Stateless sessions:** ``SessionCreationPolicy.STATELESS`` + CSRF disabled (no cookies = no CSRF surface for the token path).
- **Authorization:** two levels — (1) URL rules in ``SecurityConfig`` (public GETs vs authenticated rest), (2) object-level checks in code: only ``post.getCreator().getId().equals(userId)`` may edit/delete/respond (403 otherwise).
- **Validation:** ``@Valid`` bean validation on signup/login records (``@NotBlank``, ``@Email``, ``@Size(min = 8)``); manual required-field checks in ``PostController.create``.
- **CORS:** explicit allow-list of methods/headers with credentials — but only origin ``http://localhost:3000`` (mismatch with Vite 5173 — honest gap, Section 11).
- **H2 console** reachable only at ``/h2-console/`` **and frames allowed** ``sameOrigin`` (dev convenience).
- **Global 401 entry point** returns JSON ``{"status":401,"message":"Authentication required"}`` — never an HTML error page.

Error handling strategy

- **One shape everywhere:** ``{"status": <int>, "message": "<string>"}`` built by ``ApiExceptionHandler.build()``.
- Controllers throw ``ResponseStatusException(404/403/409/400, reason)`` — they never build error JSON themselves.
- Unhandled exceptions: logged with full stack (``log.error``), client gets a safe **500 "Internal server error"** (no leak of internals).
- Frontend: ``request()`` throws ``ApiError(status, message)``; screens render ``error.message`` inline (e.g., red text under the login form).

Performance considerations (honest state)

- **What exists:** newest-first ordering from SQL (``OrderByCreatedAtDesc``); ``UserController.list`` caps output at ``limit(100)``; recommendation endpoint sorts once in memory; frontend bundles ~327 kB JS / ~29 kB CSS (gzipped ~94 / 6.6 kB).
- **What is missing (named limitations):** no pagination on ``/api/posts``, filtering happens **in memory after** ``findAll()``, no DB indexes declared, several ``@ManyToOne(fetch = EAGER)`` relations, ``markAllRead()`` saves rows one-by-one, dashboard fires **6 parallel** ``useFetch`` calls.
- **My rule of thumb for interviews:** "correct first, then measure; pagination + indexes are the first two things I would add under load."

REMEMBER: Never claim caching, Redis, or a CDN — none exist here. Describe them as the **next step** under scale questions.

10. Testing, Deployment and DevOps

Testing that exists

- **Integration tests:** ``backend/src/test/java/com/teamup/TeamupApiTest.java`` — ``@SpringBootTest`` + ``@AutoConfigureMockMvc`` (full app context, real H2, real security filter chain, no browser).
- **Test 1** — ``loginSucceedsAndWrongPasswordFails``: valid seeded user gets a token; wrong password must return **401**.
- **Test 2** — ``nonCreatorCannotPatchPost``: Samira PATCHes Maya's post -> expects **403** (authorization proof).
- **Test 3** — ``joinRequestLoopAcceptAddsMember``: submit (201) -> duplicate (409) -> creator lists requests -> accept -> member added (state machine proof).
- **Style:** Arrange-Act-Assert with a ``login()`` helper that extracts ``accessToken`` from the response JSON; assertions via AssertJ + MockMvc status matchers.
- **Run:** ``./mvnw test`` (Maven wrapper, no Maven install needed).
- **Not found in repo:** frontend unit tests (no test script in ``package.json``), no coverage tooling, no E2E (Cypress/Playwright).

How it is run (the "deployment" story)

```
Backend: cd backend && ./mvnw spring-boot:run      -> http://localhost:4000
Frontend: pnpm install && pnpm dev                  -> http://localhost:5173
Production frontend: pnpm build -> dist/ ; pnpm preview (static server)
DB: H2 in-memory, auto-created by Hibernate (create-drop), seeded by DataSeeder
```

- **Environment configuration:** ``APP_JWT_SECRET``, ``APP_JWT_EXPIRATION_MS`` (backend, via ``application.yml`` ``${...}`` defaults); ``VITE_API_URL`` (frontend, defaults to ``http://localhost:4000``).
- **Secrets hygiene:** ``.env*`` files are git-ignored; the yml ships a **weak dev default secret** — I would fail fast in prod if the env var is missing.

CI/CD

- **Not found in repo:** no ``.github/workflows``, no Dockerfile, no docker-compose, no deployment scripts.

INTERVIEW TIP: If asked about CI/CD, describe what you *would* wire, confidently and concretely: "GitHub Actions: on push -> ``mvnw test`` + ``pnpm build``, then Docker image for the API and static deploy of ``dist/`` to a CDN." Never pretend it exists.

11. Limitations, Improvements and Future Scope

Honest weaknesses in the current code

- **In-memory H2 + `create-drop`** — every restart wipes all data; no Flyway/Liquibase migrations in the repo.
- **No service layer** — business rules (guards, notifications, member updates) live inside controllers, which limits reuse and unit testing.
- **CORS origin is `localhost:3000` but Vite serves 5173** — cross-origin calls from the dev server would be blocked; needs `http://localhost:5173` (or a Vite proxy). A real bug I would fix first.
- **No pagination / DB-side filtering** — `PostController.list()` calls `findAll()` then filters in Java; fine for hundreds of rows, not for 100k+.
- **No indexes declared; no unique constraint on `users.email`** — signup duplicate check is an app-level `existsByEmail()` (race window).
- **No `@Transactional` on multi-write flows** — `respond()` writes request + post + notification; a crash midway can leave partial state.
- **EAGER `@ManyToOne` everywhere** — listing requests builds full nested `PostView`s; classic N+1 / over-fetch territory at scale.
- **Token in localStorage** — readable by any injected script (XSS). HttpOnly cookies or short-lived + refresh tokens are the usual upgrades.
- **No refresh token / no 401 auto-logout** — after 15 minutes the user's next action fails; the SPA does not proactively re-authenticate.
- **Forgot-password and teams screens are UI-only** — no backend flow exists ("Not found in repo" for email reset).
- **No rate limiting, no audit logs, no HTTPS story** — acceptable for a demo, not for production.

How I would scale to 1 lakh (100k) / 1 million users

- **DB first:** move to PostgreSQL; add indexes (`users.email` unique, `posts(created\_at desc)`, `posts(status)`, `join\_requests(requester\_id)`); enable pagination (`Pageable`, keyset pagination on `created\_at`); push filters into SQL (Specifications or JPQL) instead of `findAll()` + streams.
- **Read path:** CDN + cached static SPA; Redis cache for the hot feed with short TTL; ETags/`If-None-Match` for repeated GETs.
- **Write path:** extract services with `@Transactional`; asynchronous notifications (event queue) so a join request is not slowed by notification inserts.
- **Stateless horizontal scale:** JWT already fits — put N instances behind a load balancer; move sessions/tokens to short-lived + refresh rotation; secrets in a vault.
- **Frontend:** code-split routes (`React.lazy`), paginate the feed, batch the dashboard's 6 calls into one aggregate endpoint.
- **Ops:** Docker images, GitHub Actions pipeline, health/readiness probes (`/api/health` already exists), metrics + structured logs, rate limiting at the gateway.
- **Numbers to quote:** 100k users x ~20 requests/day ≈ 2M req/day ≈ ~25 rps average — a single well-tuned instance handles that; the real pressure is feed reads, which caching absorbs.

If I rebuilt it from scratch

- Service layer + DTO mappers from day one; `@Transactional` on every multi-write use case.
- PostgreSQL + Flyway migrations committed with the code (schema in git, not implicit in entities).
- Pagination + DB indexes from the first commit — cheap now, painful to retrofit.
- Unique email constraint + proper 409 handling; refresh-token auth in HttpOnly cookies.
- Same tests, plus frontend tests for `api.js` and one Playwright happy-path test.

INTERVIEW TIP: Weaknesses are not confessions — they are proof of judgment. Always pair each weakness with the concrete fix (above), so the interviewer hears "diagnosed + knows the remedy".

12. Interview Q&A Bank

INTERVIEW TIP: How to use: answer each question ALOUD in 30-45 seconds using only the bullets. Then read the follow-ups and answer those too. Every answer here is grounded in real classes and files from this repo — verify anything you quote by opening the file once before the interview.

a) Project overview and motivation

Q1. Tell me about yourself and this project.

Answer:

I am a Java full-stack developer. My project is **TeamUp**, a platform where students post projects and form teams with the right skills.

I built the **React (Vite + Tailwind) frontend** with 14 screens and the **Spring Boot REST API** with JWT auth, JPA, and H2.

The full loop works: signup -> publish post -> join request -> accept/reject -> notifications.

Follow-up you may get:

- Which part was hardest?
- Did you build it alone?

Q2. Why did you choose this project idea?

Answer:

Team formation is a **real, relatable problem** — hackathon teams form over chat and fall apart because nobody knows who is free or skilled for what.

It also forced me to cover every full-stack base: **auth, CRUD, relationships, authorization, and a polished UI** — not just a to-do app.

Follow-up you may get:

- Who is the target user?
- Is it deployed?

REMEMBER: Anchor on the word "coverage" — managers want to know you chose a project that taught you the whole stack.

Q3. What was your role and team size?

Answer:

Solo developer for both backend and frontend (repo shows one author across 13 commits).

I designed the schema, wrote the API, built all 14 screens, and wrote the integration tests.

Follow-up you may get:

- How did you manage scope?
- Would you work differently in a team?

REMEMBER: If you actually worked in a team, replace this with the truth and be ready for follow-ups about git workflow and code review.

Q4. How long did it take?

Answer:

The git history shows **13 focused commits in the delivery phase**, built feature-block by feature-block.

Most of the effort went to the design of the API contract and the request/accept state machine; pages then moved quickly against that contract.

Follow-up you may get:

- What took the longest?
- How did you keep commits clean?

Q5. What are the main features?

Answer:

Auth (signup/login with JWT), **project posts** (CRUD with roles and skills), **join requests** with accept/reject, **comments**, **notifications**, **dashboard**, **profiles** with skills/links, and a **people directory** with filters.

Plus skill-based **recommended posts** — `PostController.recommended()` scores posts by skill overlap.

Follow-up you may get:

- Which feature is most proud of?
- What is unfinished?

Q6. What makes your project different from a normal CRUD app?

Answer:

It has a **state machine**: join requests move pending -> accepted/rejected with guards (409 for duplicates, 403 for non-owners).

It has **two-sided effects**: accepting one request mutates the post (member list) AND notifies the other user.

And **object-level authorization** — not just "is logged in", but "is this user the creator of THIS post".

Follow-up you may get:

- Where is that check in the code?
- What happens on double accept?

Q7. What did you learn building it?

Answer:

How **Spring Security really works** — filter chain, SecurityContext, and why stateless JWT means the server keeps no session.

How **Hibernate maps relationships** — `@OneToMany` with cascade vs `@ElementCollection` value objects.

And **API design discipline** — one error shape `{status, message}` so the frontend has a single error path.

Follow-up you may get:

- Any mistake you would fix?
- What would you learn next?

Q8. Is this a real product? Would you deploy it?

Answer:

It is a **working full-stack application**, demo-ready locally: H2 + `DataSeeder` means anyone runs it in minutes.

For production I would swap H2 for PostgreSQL, fix the CORS origin, add HTTPS and a real deployment pipeline — those are named in my improvements list.

Follow-up you may get:

- What would the first production change be?

b) "Why" questions (stack, DB, architecture, choices)

Q9. Why Spring Boot for the backend?

Answer:

Convention over configuration: embedded server, dependency injection, security, and data access are wired by starters — I write business code, not plumbing.

It is also the **industry standard** for Java roles, so skills transfer directly to work.

Alternatives like raw servlets would mean hand-wiring every filter and datasource.

Follow-up you may get:

- What does Spring Boot auto-configure for you?
- Why not Node/Express?

Q10. Why JWT instead of server sessions?

Answer:

The API is **stateless** (`SessionCreationPolicy.STATELESS`) — any server instance can serve any request without a shared session store, which is what you need to scale horizontally.

The token carries ``sub`` (userId), ``role``, ``email`` — the filter authenticates **without a DB hit per request**.

Trade-off I accept: tokens cannot be revoked before expiry (15 min window keeps that small).

Follow-up you may get:

- What is the downside of JWT?
- How would you log a user out early?

REMEMBER: Always answer "why X" with "vs Y + trade-off". That is the shape managers listen for.

Q11. Why H2 and not MySQL/PostgreSQL?

Answer:

For this project: **zero-setup demo** — H2 runs in memory, Hibernate creates the schema (``ddl-auto: create-drop``), ``DataSeeder`` fills demo data on boot.

Spring Data JPA abstracts the DB, so switching to PostgreSQL is a **datasource change + driver**, not a rewrite.

I am clear that H2 is a **demo/dev choice, not a production choice**.

Follow-up you may get:

- What changes when you move to PostgreSQL?
- What is ddl-auto create-drop?

Q12. Why a REST API instead of server-rendered pages?

Answer:

A clean **JSON contract** lets the SPA (and later a mobile app) consume the same API.

It separates concerns: backend owns data + security, frontend owns UX.

For an auth-heavy app without SEO needs, a **pure SPA + REST** is simpler than SSR.

Follow-up you may get:

- When would you have chosen SSR?
- How does the frontend find the API URL?

Q13. Why Tailwind CSS?

Answer:

Styles live **next to the markup** — no class-name sprawl across CSS files, and design tokens are centralized in ``src/index.css`` via Tailwind v4 ``@theme`` (ink, paper, orange...).

Consistency became a token: same breakpoint scale, same colors on all 14 screens.

I tried the alternative (plain CSS modules) earlier — keeping 14 screens visually consistent got messy.

Follow-up you may get:

- What is Tailwind v4 `@theme`?
- Any downsides of utility classes?

Q14. Why Vite (or: why not Next.js)?

Answer:

Vite gives **instant dev start and fast builds** with almost no config; the app is a client-rendered dashboard, so SSR adds complexity without SEO benefit.

Next.js would add a server runtime and routing model I did not need — **auth-gated screens are not crawled by search engines** anyway.

React Router handles the 14 client routes cleanly.

Follow-up you may get:

- What would make you move to Next.js?
- How big is your production bundle?

Q15. Why did you write a custom `api.js` instead of Axios?

Answer:

The app needs exactly three things: **base URL + JSON + Bearer token + one error format** — ``fetch`` gives that in ~40 lines.

One wrapper means **one place to change** auth or error handling; every screen uses ``api.getPosts()``-style helpers, never raw fetch.

Axios would be fine too — interceptors do the same job — but adding a dependency for 40 lines was not worth it.

Follow-up you may get:

- What does your wrapper do on 401?
- Why a custom hook for fetching?

Q16. Why is there no Redux / state library?

Answer:

Server state is handled by ``useFetch`` + `refetch`; UI state is local ``useState``; the token lives in `localStorage`; saved posts in `localStorage` (``saved.js``).

There is **no complex shared client state** that justifies a global store — adding one would be ceremony.

If state grew (filters, notifications, optimistic updates), I would introduce a store or a server-state library like React Query.

Follow-up you may get:

- When would you add Redux?
- How do pages get fresh data after an action?

Q17. Why did you put business logic in controllers instead of services?

Answer:

Honest answer: **scope control** — for a project this size, a service layer would have added classes without changing behavior much.

I know the cost: logic is harder to reuse and to unit-test in isolation. **`PostService` is the first refactor I would do.**

The controller methods are still small because rules are extracted into helpers like ``applyPostFields()`` and ``currentOwnerPost()``.

Follow-up you may get:

- Show me where you would cut the boundary?
- How would you test services?

REMEMBER: This is a trap question. Never defend the choice as ideal — present it as a conscious trade-off with a named refactor.

Q18. Why store enums as strings in the DB?

Answer:

``@Enumerated(EnumType.STRING)`` keeps rows readable ("Open" instead of 0) and **survives reordering or inserting enum constants** — ordinals shift and silently corrupt data.

Cost: a few more bytes and a slightly larger WHERE clause; correctness wins.

Follow-up you may get:

- What is the ordinal alternative risk?
- Any query impact?

c) "How" questions (flows and behavior)

Q19. How does login work end to end?

Answer:

Frontend `login.jsx` posts email+password to `/api/auth/login`.

`AuthController.login()` looks up the user (lowercased email), calls `passwordEncoder.matches()`; failure returns **401 with a generic message**.

Success: `jwtService.generateToken(id, role, email)` returns a signed 15-minute token; response is `{user, accessToken}`.

Frontend stores it in **localStorage** (`teamup_token`) and every later call sends `Authorization: Bearer <token>`.

Follow-up you may get:

- Where is the password stored?
- What happens when the token expires?

Q20. How does the server know a request is authenticated?

Answer:

`SecurityConfig` registers `JwtAuthFilter` **before** `UsernamePasswordAuthenticationFilter` in the chain.

The filter reads the Bearer header, `jwtService.isValid()` checks signature + expiry, and on success it sets an authenticated `UsernamePasswordAuthenticationToken` (principal = `userId`) into `SecurityContextHolder`.

Controllers then read `Authentication` and cast the principal to `Long`.

Follow-up you may get:

- What if the header is missing?
- What if the token is expired?

REMEMBER: Mention "filter chain" by name — it signals you know Spring Security is a chain, not a single check.

Q21. How is data stored (what happens when I save a post)?

Answer:

`postRepository.save(post)` hands the entity to Hibernate; on **first persist** `@PrePersist` sets `createdAt/updatedAt`.

Hibernate inserts the `posts` row and, because `rolesRequired` has `cascade = ALL`, also inserts each `post_roles` row in the same unit of work.

Everything is one logical save from the controller's point of view.

Follow-up you may get:

- What does `orphanRemoval` do?
- Which DB tables are touched?

Q22. How does the join-request flow handle edge cases?

Answer:

Requesting **your own post** -> 400 ("cannot request to join your own post").

Duplicate pending request -> 409 CONFLICT (checked in `submit()` by scanning the requester's requests).

Responding to an **already handled** request -> 409 ("already been responded to").

Someone **else** trying to respond -> 403 via `currentOwnerPost()` owner check.

Follow-up you may get:

- Where exactly is each check?
- What if two accepts race?

Q23. How do notifications get created?

Answer:

Inside `JoinRequestController`: after a successful submit/accept/reject it calls the private `notify(user, type, message, post, request)`, which saves a `Notification` row (`read=false`).

Types are an enum: `joinRequest`, `requestAccepted`, `requestRejected`, ...

The recipient sees them at `GET /api/me/notifications`, ordered newest-first by the derived query `findByUserIdOrderByCreatedAtDesc`.

Follow-up you may get:

- Any push/WebSocket? (No — polling on page load)
- How is "mark all read" done?

Q24. How does the frontend get and display data (useFetch)?

Answer:

Pages call `useFetch(() => api.getPosts(), [])` which returns `{data, loading, error}`.

The hook runs the promise in `useEffect`, sets `loading`, stores the result, and uses an `active` flag so a response arriving after unmount does not setState (no React warning, no stale write).

After mutations (comment, join), pages call the api again to refetch.

Follow-up you may get:

- Why deps array?
- How does it avoid race conditions?

Q25. How does search/filtering work on the feed?

Answer:

Two layers: the API **accepts** `category/mode/status/search` params and filters in Java after `findAllOrderByCreatedAtDesc`.

The `posts.jsx` page loads the list once and filters **client-side in `useMemo`** as you type — instant, zero network.

So typing is free, and the server params exist for other clients (curl, mobile).

Follow-up you may get:

- Is that scalable? (See Section 11 — no)
- Why filter twice?

Q26. How does authorization differ from authentication?

Answer:

Authentication = who are you: JWT filter sets the principal from a valid token.

Authorization = what may you do: two levels here — URL rules in `SecurityConfig` (public GETs vs authenticated rest) and object checks in code (`post.getCreator().getId().equals(userId)` else 403).

The test `nonCreatorCannotPatchPost` proves the second level works.

Follow-up you may get:

- Where would you use `@PreAuthorize`?
- What does the URL rule alone not protect?

Q27. How do you create the demo data?

Answer:

`DataSeeder` implements CommandLineRunner` — Spring Boot calls run() at startup.`

It guards with `if (userRepository.count() > 0) return;` so it seeds **once**, and is `@Transactional` so the seed is atomic.`

It creates 5 users (Samira, Aarav, Maya, Rohan, Nisha — all `@teamup.dev`), 4 posts with roles, comments, requests, notifications; password teamup-demo-2026` (bcrypt-hashed).`

Follow-up you may get:

- Why `CommandLineRunner`?
- How do tests depend on it?

Q28. How do you debug a failing API call?

Answer:

Reproduce **outside the browser first**: `curl` the exact endpoint with/without the token to see the raw status + body.`

Read the server log — `ApiExceptionHandler` logs unhandled exceptions with full stack (log.error`).`

Then narrow: 401 -> filter/token; 403 -> owner check; 409 -> guard; 400 -> validation; 500 -> stack trace.

Finally, write a **MockMvc test** reproducing it so it stays fixed.

Follow-up you may get:

- How do you know it is frontend vs backend?
- What tools do you use?

Q29. How does the app handle errors on the frontend?

Answer:

`request()` throws ApiError(status, message)` parsed from the server body.`

Screens catch it in the submit handler and show `error.message` inline (e.g., red text under the login form) — no alert() popups.`

`useFetch` exposes an error` state for load failures.`

Follow-up you may get:

- What happens if the backend is down?
- How do you show network vs API errors?

Q30. How would you add a new endpoint — walk me through it.

Answer:

1. **Entity/repo**: add a method to the right `JpaRepository` (or reuse one).`

2. **Controller**: new method with mapping + `Authentication` if protected; throw ResponseStatusException` for failures.`

3. **Authorization**: check if the URL needs a new rule in `SecurityConfig` (default is authenticated).`

4. **View record:** shape the JSON with a `XxxView.from()` mapper.
5. **Frontend:** add an `api.xxx()` helper, call it with `useFetch`, render.
6. **Test:** one MockMvc test for the happy path + one for the denial.

Follow-up you may get:

- What if it changes the DB schema?
- Where do you document it?

d) Code-level questions (specific classes in this repo)

Q31. Explain `PostController.update()` — who may edit and how?

Answer:

It loads the post (`findOrThrow` -> 404 if missing`), gets `userId`` from the ``Authentication`` principal (401 if absent). Then the **owner check**: ``if (post.getCreator() == null || !post.getCreator().getId().equals(userId)) throw 403 "Only the creator can edit this post"``.

Fields are patched **only when non-null** (``if (request.title() != null)...``) plus ``applyPostFields()`` for `deadline/mode/status/roles/members`.

Finally ``postRepository.save(post)`` and the response is a ``PostView``.

Follow-up you may get:

- Why PATCH not PUT?
- What if creator is null?

Q32. What does `applyPostFields` do and why is it separate?

Answer:

It copies optional fields from the request DTO onto the entity: `deadline`, `mode`, `status`, `links`, and it **rebuilds the ``rolesRequired`` and ``currentMembers`` lists** into managed entities.

Separate because create AND update share it — one place defines "which fields a post accepts" (DRY).

Note: when ``rolesRequired`` is sent, the whole list is replaced; ``cascade=ALL + orphanRemoval`` cleans old rows.

Follow-up you may get:

- What risk does replacing the list carry?
- How are `PostRole` children linked?

Q33. `JwtService.isValid()` returns false for everything bad — is that OK?

Answer:

It catches all exceptions (bad signature, expired, malformed) and returns false — **safe default: deny**.

The filter then simply does not set authentication, so protected endpoints 401.

Trade-off: the reason is swallowed. For observability I would log the failure type (expired vs invalid) at debug level.

Follow-up you may get:

- How would you distinguish expired from invalid?
- Where is the 401 actually produced?

Q34. Why does `JwtAuthFilter` always call `filterChain.doFilter()`?

Answer:

Because the filter must **not break public endpoints** — ``/api/health`` and ``/api/auth/`` **must work without a token**.

If the token was invalid, the `SecurityContext` simply stays empty; ``authorizeHttpRequests`` rules decide what happens next (401 entry point for protected URLs).

A filter that threw or short-circuited would kill every request.

Follow-up you may get:

- What does the entry point return?
- Where is it configured?

Q35. What is the purpose of `UserView` / `PostView` records?

Answer:

They are **response DTOs**: `UserView.from(user)` turns an entity into a flat JSON-safe shape.

Benefits: hide `password`, control field names, and avoid returning JPA internals (proxies, lazy collections) directly.

They also decouple API shape from DB shape — I can change an entity without breaking clients.

Follow-up you may get:

- Where is the password excluded?
- Difference vs entities?

Q36. Explain `ApiExceptionHandler.handleGeneric` — what does the client see?

Answer:

Any exception no more specific handler claims lands here: it **logs the full stack server-side** and returns HTTP 500 with `{"status":500,"message":"Internal server error"}`.

Purpose: never leak stack traces, SQL, or class names to the client while keeping ops debugging possible from logs.

Follow-up you may get:

- Why not return the exception message?
- How does the frontend render it?

Q37. What does `MeController.markAllRead()` do — and its inefficiency?

Answer:

It loads all notifications for the user, flips `read=true` on unread ones, and **saves each row individually**, returning `{updated: count}`.

Inefficiency: N separate UPDATE statements. A single `UPDATE notifications SET read=true WHERE user_id=? AND read=false` (derived/`@Modifying` query) would be one round trip.

I know it — it is on my improvement list.

Follow-up you may get:

- How would you write that query?
- Why not `@Transactional` here?

Q38. `CurrentUser` helpers are repeated in 4 controllers — what do you think?

Answer:

It is **duplication I would extract** into a small utility or a `@AuthenticationPrincipal`-style resolver (e.g., a custom `HandlerMethodArgumentResolver` that injects the current `User`).

Why it happened: keeping each controller self-contained while moving fast.

Same for `findOrThrow` — a `NotFound` helper or `@ControllerAdvice` model binding would dedupe it.

Follow-up you may get:

- What is an argument resolver?
- Any other duplication you see?

REMEMBER: Interviewers deliberately hunt for duplicated code. If you point it out BEFORE they do, you win the point.

Q39. Why is `Post.creator FetchType.EAGER`?

Answer:

Because almost every response needs the creator (name, id) — EAGER avoids an N+1 when serializing a list of posts to `PostView`.

The flip side: any query loading posts also loads users — fine at this size, but under load I would switch to **JOIN FETCH in the query** or DTO projections to control it explicitly.

Follow-up you may get:

- What is N+1 exactly?
- When is EAGER dangerous?

Q40. What does the `@Valid` annotation actually trigger?

Answer:

On `AuthController.signup(@Valid @RequestBody SignupBody body)`, Spring runs **Bean Validation** on the record: `@NotBlank name`, `@Email email`, `@Size(min=8) password`.

Failures throw `MethodArgumentNotValidException`, which `ApiExceptionHandler` converts to **400 with the first field message**.

No `@Valid` = the method runs with bad data — which is why other endpoints do manual checks.

Follow-up you may get:

- What status code for validation failure?
- Why records for DTOs?

Q41. How does the frontend keep the token — and what would you change?

Answer:

On login/signup, `setToken(accessToken)` writes it to **localStorage**; `getToken()` reads it; `request()` attaches it as a Bearer header.

What I would change: **HttpOnly Secure cookie** (immune to XSS theft) plus refresh-token rotation, accepting the CSRF mitigations that requires — or short-lived access tokens in memory only.

Right now localStorage is the pragmatic SPA choice, and I state its trade-off openly.

Follow-up you may get:

- What is XSS and how does localStorage enable it?
- Why not put JWT in a cookie as-is?

Q42. Walk me through `src/lib/api.js` — the `request()` function.

Answer:

Builds headers (`Content-Type: application/json` + optional `Authorization: Bearer`).

`fetch(API_URL + path, {method, body, headers})` — API_URL from `VITE_API_URL`, default `http://localhost:4000`.

On `!response.ok`: parses `{message}` and **throws `ApiError(status, message)`**; on 204 returns undefined, else parses JSON.

Everything else (`api.getPosts`, `api.login`...) is a typed wrapper over this one function.

Follow-up you may get:

- Why lowercase the email before login?
- How do you switch environments?

e) Database questions (schema, joins, transactions, indexes)

Q43. Walk me through your schema.

Answer:

Six entities: **users**, **posts**, **post_roles**, **join_requests**, **comments**, **notifications**.

`posts.creator_id` -> users; `join_requests` has BOTH `post_id` and `requester_id`; `comments` has `post_id` + `author_id`; `notifications` points to a user plus optional post/request.

Collections (skills, interests, timeline, current members, role skills) live in **side tables** created by `@ElementCollection`.

Follow-up you may get:

- What is the table for members called?
- Why is PostRole a full entity?

Q44. Why is currentMembers an @ElementCollection but rolesRequired an entity?

Answer:

`currentMembers` is a **snapshot for display** (userId + userName + role + joinedAt) — no behavior, no independent lifecycle -> value object fits.

`rolesRequired` is a **managed child**: it has an id, is updated when the creator edits roles, and needs `orphanRemoval` to delete removed rows.

Rule I applied: "does it need its own identity and independent updates?" yes -> entity.

Follow-up you may get:

- What problem would a member entity solve?
- What does orphanRemoval = true do?

Q45. What relationships exist, and which side owns the FK?

Answer:

All the "many" sides own FKs via `@JoinColumn`: `posts.creator_id`, `join_requests.post_id`, `join_requests.requester_id`, `comments.post_id`, `comments.author_id`, `notifications.user_id`.

So the DB is classic **many-to-one from child to parent**; the OneToMany sides are the inverse (`mappedBy = "post"` on roles).

Follow-up you may get:

- Which is the inverse side?
- How does mappedBy avoid duplicate columns?

Q46. Do you have indexes? Where do you need them?

Answer:

In the repo: **none declared** (no `@Index`, no migration DDL) — only PK auto-indexes.

Under load I would add: **unique index on users.email** (also fixes the signup race), composite **posts(created_at desc, status)** for the feed, and **join_requests(requester_id) / (post_id, status)** for the request queries.

Every list method ends with `OrderByCreatedAtDesc` — that ordering column is exactly what a composite index should cover.

Follow-up you may get:

- What is a covering index?
- How do you verify an index is used?

REMEMBER: Saying "none — and here is exactly where I would start" is a strong, honest answer.

Q47. Is your schema normalized? Any denormalization on purpose?

Answer:

Mostly **3NF**: attributes depend on the key (user skills in a child table, roles as their own table).

Deliberate denormalization: **member snapshot stores `userName`** (avoids a join and survives renames) and token claims carry `role`/`email` (avoids a DB read per request).

Both are trade-offs I chose for read simplicity.

Follow-up you may get:

- What are the denormalization risks?
- How do you keep snapshots in sync?

Q48. Where are transactions used? What is missing?

Answer:

``DataSeeder.run()`` is ``@Transactional`` — the whole seed is all-or-nothing.

Missing: ``JoinRequestController.respond()`` writes request + post + notification in three separate implicit transactions. A crash mid-way could leave a partial state.

Fix: annotate the method ``@Transactional`` (or move it into a service) so all writes commit together.

Follow-up you may get:

- What is the default isolation level?
- What is a dirty read?

REMEMBER: This is a favorite "do you actually know transactions" probe. Lead with the missing `@Transactional` — it shows depth.

Q49. How would you paginate the feed — concretely?

Answer:

Backend: ``Page<Post> findAll(Pageable pageable)`` or a derived ``findTop20ByStatusOrderByCreatedAtDesc(...)``; return ``Page<PostView>`` with total elements.

Keyset pagination (cursor = last ``created_at,id``) beats OFFSET at scale because OFFSET still scans skipped rows.

Frontend: "Load more" appending pages; keep the client-side ``useMemo`` filters for the page already loaded.

Follow-up you may get:

- OFFSET vs keyset cost?
- How does the API shape change?

Q50. H2 vs PostgreSQL — what actually changes in your code?

Answer:

Config: datasource URL/driver/credentials in ``application.yml`` + Postgres driver dependency.

Schema: replace ``ddl-auto`` with **Flyway migrations** (H2/Postgres SQL differences bite if Hibernate guesses).

Code: mostly identical because JPA abstracts it; watch ``ILIKE`` vs ``LIKE`` for case-insensitive search, and add real indexes.

Follow-up you may get:

- What is Flyway?
- What breaks in SQL dialects?

Q51. How do derived query methods work (`findByUserIdOrderByCreatedAtDesc`)?

Answer:

Spring Data **parses the method name at startup** into a query: entity path ``user.id = ?1 ORDER BY created_at DESC`` — no handwritten SQL.

It fails fast at boot if the property does not exist (typo-proofing).

For anything more complex (joins, aggregates, updates) I would use ``@Query`` JPQL or Specifications.

Follow-up you may get:

- What if the query gets too complex?
- What is `@Modifying`?

f) API and backend design questions

Q52. How did you design your API? What conventions did you follow?

Answer:

Noun-based REST paths (`/api/posts`, `/api/posts/{id}/comments`) with HTTP verbs for actions (GET read, POST create, PATCH partial update, DELETE remove).

Consistent JSON: resources return View records; errors always `{status, message}`.

Auth semantics: 401 = not authenticated, 403 = wrong user, 404 = missing resource, 409 = state conflict, 400 = bad input, 201 on create, 204 on delete.

Follow-up you may get:

- Why PATCH for updates?
- GET /api/users returns a list — is that RESTful?

Q53. Why is GET /api/posts public but POST not?

Answer:

It is a **product decision encoded in SecurityConfig**: the feed is the landing experience (browse before signup), while writing requires identity.

Rule: ``requestMatchers(HttpMethod.GET, "/api/posts/", "/api/users/").permitAll();``
``anyRequest().authenticated();`` covers writes.

Note the consequence: public reads mean personal data must not leak in list payloads — `UserView` excludes the password and only exposes profile fields.

Follow-up you may get:

- Is that safe for private posts? (No private posts exist)
- How would you add private posts?

Q54. Why 409 on duplicate join request and not just ignore it?

Answer:

The client asked to create something that **already exists in a conflicting state** — 409 CONFLICT is the semantically correct code.

It also gives the UI a real message to show ("You already have a pending request for this post") instead of silently double-writing.

Ignoring it would hide bugs and let duplicate rows accumulate.

Follow-up you may get:

- What other 409 cases exist?
- Difference between 409 and 400?

Q55. Why is there no service layer — where would you put business rules?

Answer:

Current shape: Controller (HTTP) -> Repository (SQL). Rules like ownership checks live in controller helpers (`currentOwnerPost`, `applyPostFields`).

If the app grew, I would extract **PostService / JoinRequestService**: controllers become thin (parse + delegate), rules become unit-testable without MockMvc, and `@Transactional` sits on the service method.

I would keep repositories as the only SQL-facing layer (same pattern, one extra hop).

Follow-up you may get:

- How does the test change with services?
- Where would validation live?

Q56. How do you version or evolve the API without breaking clients?

Answer:

Current repo has no version prefix; for a real rollout I would use `/api/v1/...` and keep additive changes (new fields are ignored by old clients).

Breaking changes (field renames) would go to v2 with both live during migration; error shape `{status, message}` is the stable contract my frontend relies on.

Follow-up you may get:

- What counts as a breaking change?
- How do clients handle unknown fields?

Q57. How does CORS work here — and is your config correct?

Answer:

`SecurityConfig.corsConfigurationSource()` allows methods GET/POST/PATCH/PUT/DELETE/OPTIONS, all headers, credentials, and origin `http://localhost:3000`.

Honest catch: the Vite dev server runs on **5173**, so cross-origin dev calls would be rejected until I add that origin (or proxy through Vite). This is a real bug I would fix first.

In production the same bean would list the deployed frontend origin.

Follow-up you may get:

- What is a preflight request?
- Why allowCredentials with which auth style?

REMEMBER: Volunteering this bug — calmly, with the fix — reads as senior-level review skill.

Q58. What does the health endpoint do and why does it exist?

Answer:

`GET /api/health` returns `{ "service": "teamup-api", "status": "ok" }` and is `permitAll`.

Purpose: **liveness probe** for load balancers/CI/monitoring without credentials; also my quick check that the server is up before debugging anything else.

Follow-up you may get:

- Liveness vs readiness?
- What would you add (version, db status)?

Q59. How do you decide what JSON a client gets — entities or DTOs?

Answer:

Always **View records** (`PostView.from`, `UserView.from`, `CommentView.from`) — entities never leave the controller layer directly.

Example: `UserView` omits `password`; `PostView` flattens `creator` into a nested object the card component needs in one fetch.

This keeps JSON stable even if the DB schema evolves.

Follow-up you may get:

- What about write requests (incoming)?
- Where are records defined?

Q60. How would you document this API for other developers?

Answer:

README currently documents endpoints as a table — good start for a project.

Next step: **OpenAPI/SpringDoc** generated from annotations at ``/swagger-ui.html``, so docs cannot drift from code; plus the MockMvc tests as executable examples.

Follow-up you may get:

- What is OpenAPI?
- How do you keep docs in sync?

g) Frontend questions

Q61. How is the frontend structured?

Answer:

Vite + React 19 SPA: `App.jsx` defines **14 routes** with React Router (plus a catch-all `\*` -> home) and a `ScrollToTop` component.

`src/lib` holds cross-cutting code (`api.js`, `use-fetch.js`, `saved.js`); `platform-shell.jsx` gives every page the sidebar/topbar chrome; `src/pages` = one file per screen.

Design tokens and the blueprint grid live in `src/index.css` (Tailwind v4 `@theme`).

Follow-up you may get:

- Why one file per page?
- What does PlatformShell export?

Q62. How does a page fetch data? (useFetch)

Answer:

`const { data, loading, error } = useFetch(() => api.getPosts(), [])` — one line per resource.

The hook wraps the promise in `useEffect`, tracks loading/error state, and sets an **active` flag** so late responses after unmount are ignored.

The deps array (`[id]` on detail pages) re-triggers the fetch when the route changes.

Follow-up you may get:

- What problem does the active flag solve?
- How do you refetch after a mutation?

Q63. How does routing work without page reloads?

Answer:

React Router v7 intercepts link clicks, updates the **URL via history API**, and renders the matching component — the page never reloads, so state (like the token) survives.

`ScrollToTop` resets scroll on pathname change because SPA navigation keeps the old scroll position.

Deep links like `/posts/2` work because Vite serves `index.html` for any path.

Follow-up you may get:

- What happens on a hard refresh at /posts/2?
- How does this differ from MPA?

Q64. How does the login page keep you logged in after refresh?

Answer:

The JWT is written to **localStorage** on login; on the next session `getToken()` reads it and `request()` re-attaches the header.

The server is **stateless** — it does not remember you; every request re-proves identity with the token.

Consequence: logging out = removing the key (`setToken(null)`); no server call needed.

Follow-up you may get:

- What are localStorage's security limits?
- What happens after 15 minutes?

Q65. How do forms and validation work on the client?

Answer:

Controlled inputs via `useState` (email, password, error, submitting) and `handleSubmit` prevents default.

Client checks are for **UX** (empty fields, matching passwords); the **server is the authority** — `@Valid` on signup and manual checks reject bad data with 400/409.

Server messages are displayed inline from `ApiError.message`.

Follow-up you may get:

- Why not trust client validation?
- How is double-submit prevented?

Q66. How is the UI styled consistently across 14 pages?

Answer:

Tailwind utility classes + shared tokens: `ink #17233d`, `paper #f3efe4`, `orange #e56e3c` defined once in `@theme` and referenced everywhere.

Reusable components (`PlatformShell`, `SectionCard`, `Avatar`, `PageButton`) prevent copy-paste drift; custom utilities like `blueprint` give the signature grid-paper look.

Responsive via `max-[1050px]:` and `max-[720px]:` variants.

Follow-up you may get:

- What is Tailwind `@theme`?
- How do you keep long `className` lists readable?

Q67. The dashboard fires 6 requests on load — is that OK?

Answer:

Functionally fine (parallel, independent), and each is `useFetch` so they render as they arrive.

At scale I would **aggregate server-side** into one `/api/me/overview` returning posts+requests+notifications in a single round trip, cutting latency and server work.

Follow-up you may get:

- What is the waterfall risk?
- Would you use GraphQL here?

Q68. How do you handle loading and empty states?

Answer:

`useFetch` exposes `loading` — pages show skeleton/placeholder text until data lands.

Empty states are explicit (e.g., "no requests yet") instead of blank screens; errors render inline messages.

Mutations use a `submitting` flag to disable buttons (prevents double POSTs).

Follow-up you may get:

- How do you avoid layout jumps?
- What about optimistic UI?

Q69. Why JavaScript (.jsx) and not TypeScript — did you consider it?

Answer:

The final stack is plain JS for **speed of delivery and zero build friction**; the API DTOs (`PostView`, `UserView` in `api.js`) document the contract in JSDoc-style shapes.

I know what I traded: compile-time safety on the client. On a bigger team I would adopt TypeScript — the migration is mechanical for a codebase this size.

Follow-up you may get:

- How do you catch API shape bugs then?
- What would TS give you concretely?

REMEMBER: Do not apologize for JS. State the trade-off and the migration path confidently.

h) Security questions

Q70. How are passwords stored?

Answer:

BCrypt with cost 12 (``new BCryptPasswordEncoder(12)`` in ``SecurityConfig``) — a salted, slow hash; the raw password is never stored or logged.

Signup: ``passwordEncoder.encode(raw)``; login: ``passwordEncoder.matches(raw, storedHash)``.

Cost 12 = 2^{12} rounds — deliberately slow to brute-force, still ~100ms per login (a good balance).

Follow-up you may get:

- Why not MD5/SHA-256?
- What is a salt?

Q71. Is your JWT secure? What is in it?

Answer:

Contents: ``sub`=userId, `role`, `email`, `iat`, `exp` (15 min)` — **no password or secrets inside** (payload is only base64-encoded, not encrypted).

Signature: HMAC-SHA with ``app.jwt.secret`` (env-overridable) — tampering invalidates the signature.

Weakness I acknowledge: **HS256 shares one secret** between auth server and APIs; for multiple services I would move to RS256 (public/private keys).

Follow-up you may get:

- Can someone read a JWT?
- What happens if the secret leaks?
- Explain the alg confusion attack.

Q72. Why is CSRF disabled — is that safe?

Answer:

CSRF exploits **cookie-based** session auth: browser auto-attaches cookies, so a malicious site can forge requests.

Here auth rides in an **Authorization header from JS** — a cross-site page cannot set that header (and there is no session cookie), so the CSRF surface is gone.

If I moved the JWT into a cookie, I would have to **re-enable CSRF protection** (or use `SameSite=strict` + double-submit tokens).

Follow-up you may get:

- What attacks does this design still allow?
- What is CORS's role vs CSRF?

REMEMBER: This exact answer (CSRF off because auth is header-based) is a top-tier security response.

Q73. How do you prevent user enumeration?

Answer:

Login returns the **same 401 "Invalid email or password"** for unknown email and wrong password — no signal which one failed.

Signup is different (409 "Email already registered") — a deliberate UX trade-off; a stricter design would use email verification instead of revealing it.

Follow-up you may get:

- Is 409 on signup a leak?
- How would you redesign signup?

Q74. Explain your authorization model — how deep does it go?

Answer:

Level 1 — **URL rules**: health/auth/public GETs open, everything else authenticated.

Level 2 — **ownership in code**: edit/delete post, list/respond to requests all verify ``creator.id == userId`` (403 otherwise).

Level 3 — **data scoping**: ``/api/me/*`` endpoints only ever return rows tied to the principal's id.

Follow-up you may get:

- Why not `@PreAuthorize` annotations?
- Can user A read user B's notifications? (No — query by `user_id`)

Q75. What would you do to harden this for production?

Answer:

HTTPS everywhere; secrets in env/vault (fail fast if ``APP_JWT_SECRET`` is default); rotate signing keys.

Rate-limit auth endpoints (brute force); add refresh-token rotation; move token to `HttpOnly` cookie or keep short-lived + in-memory.

DB: unique email index, parameterized queries (JPA already), least-privilege DB user; audit logging for auth events.

Plus dependency scanning + security headers (CSP) — none are in the repo today.

Follow-up you may get:

- What is rate limiting at which layer?
- How do you rotate JWT keys?

Q76. Are SQL injections possible in your app?

Answer:

Not through JPA derived queries — they use **prepared statements / bind parameters**, never string-concatenated SQL.

There is **no native** `@Query`` with **string concatenation** in the repo, and no ``order by`` built from raw user input (sorting is fixed in the method name).

Rule: never build SQL by pasting user input; if I ever need dynamic sort fields, whitelist them.

Follow-up you may get:

- What is an ORM?
- Any place user input reaches a query? (search -> contains, still parameterized)

Q77. How do you handle logout — can the token be revoked?

Answer:

Frontend: ``setToken(null)`` removes `localStorage` — **that is the whole logout** (client-side).

Server: JWT remains crypturally valid until ``exp`` (max 15 min) because it is stateless — this is the classic JWT trade-off.

True revocation options: token blacklist in Redis, or a ``tokenVersion`` on the user checked on each request.

Follow-up you may get:

- Is 15 min short enough?
- What if the phone is stolen?

Q78. XSS — is your frontend vulnerable, and where?

Answer:

React escapes text by default (`{comment.body}` renders literally), so **stored XSS via comments is mitigated** as long as nobody uses `dangerouslySetInnerHTML` (none in the repo).

The weak spot is **localStorage holding the JWT** — any injected script could read it; mitigations: strict CSP, HttpOnly cookies, input sanitization for URLs (profile links).

Follow-up you may get:

- What is dangerouslySetInnerHTML?
- Difference XSS vs CSRF?

i) Scalability and performance questions

Q79. How would your app behave with 100,000 users?

Answer:

Bottleneck #1: **feed queries** — `findAll()` + in-memory filtering + no indexes collapses first. Fix: pagination + DB-side filters + indexes on `created_at`/`status`.

#2: **H2 in-memory** — move to PostgreSQL with connection pooling; `create-drop` also means data loss.

#3: **stateless JWT already helps** — run N Spring Boot instances behind a load balancer with no shared session state.

Frontend: route-level code splitting + CDN for `dist/`.

Follow-up you may get:

- Rough traffic math?
- What breaks first exactly?

Q80. The feed loads ALL posts — how do you fix it?

Answer:

Server: `Page<Post> findAll(PageRequest.of(page, size))` returning newest page first; filters move into a **Specification/JPQL WHERE** so the DB stops early.

Keyset cursor (`created_at < ?`) for deep scrolling — $O(1)$ per page vs OFFSET scanning.

Cache the first page (Redis or in-process Caffeine) with a short TTL — most traffic reads the same hot page.

Follow-up you may get:

- What payload shape changes?
- How does the frontend adapt?

Q81. Where are the N+1 queries in your code?

Answer:

`MeController.receiveRequests()`: loads my posts, then calls `findById...` **once per post** — classic app-level N+1.

`markAllRead()`: one `save()` per notification = N UPDATES.

Entity graph: every `@ManyToOne(EAGER)` in lists (requests -> post -> creator) adds joins; `JoinRequestView` even re-serializes full `PostView`'s.

Fixes: single JPQL with JOIN FETCH / `IN (:ids)` batch, bulk `@Modifying` update, DTO projections.

Follow-up you may get:

- How do you detect N+1 in practice? (log SQL / Hibernate statistic)
- What is JOIN FETCH?

Q82. How would you cache — and what would you NOT cache?

Answer:

Cache **read-heavy, change-tolerant** data: first feed page, popular post views (TTL 30-60s, invalidate on save).

Do NOT cache: `/api/me/*` (per-user, high cardinality + privacy), anything with live request status.

Session/token state stays stateless — caching the DB is the lever, not sessions.

Follow-up you may get:

- Cache invalidation strategy?
- What is a stampede?

Q83. How do you reduce frontend load time?

Answer:

Already: Vite build is **~327 kB JS / ~29 kB CSS (94 / 6.6 kB gzipped)**; images in ``public/`` are small placeholders.

Next: ``React.lazy`` route splitting (only load what the screen needs), preload the feed API in parallel with shell render, cache immutable assets with long cache headers via CDN.

Follow-up you may get:

- What is code splitting?
- How do you measure load time?

Q84. Is your app thread-safe / does it handle concurrency?

Answer:

Spring MVC handles requests on a thread pool; controllers are stateless (fields are dependencies) -> safe.

Real races I know about: **two accepts of the same join request** (both read ``pending``) and **double signup with same email** (exists-check then insert).

Fixes: DB unique constraint on email; re-check status inside a ``@Transactional`` with pessimistic/optimistic locking (``@Version``) on JoinRequest.

Follow-up you may get:

- What is optimistic locking?
- How would a unique constraint solve signup?

REMEMBER: Naming YOUR two known races = instant credibility. Most candidates cannot name a single one.

Q85. How would you scale notifications (say 1M events/day)?

Answer:

Today: synchronous insert inside the request path — fine for demo, adds latency under load.

Scale: emit an **event** (Kafka/RabbitMQ) after commit; consumers write notifications async; unread counts in Redis; deliver via WebSocket/SSE for real-time, with polling fallback.

Table growth: archive read notifications; index ``(user_id, read, created_at desc)``.

Follow-up you may get:

- Sync vs async trade-off?
- WebSocket vs polling?

Q86. What monitoring would you add?

Answer:

Baseline: structured JSON logs + correlation/request id; metrics (Spring Actuator + Prometheus): RPS, p95 latency, error rate by status.

Alerts on 5xx spike and p95 over SLO; ``/api/health`` as liveness, plus a readiness check that verifies DB connectivity.

Frontend: error tracking (Sentry-style) for unhandled promise rejections.

Follow-up you may get:

- What is p95?
- Which single metric do you watch first?

j) Challenges, bugs, failures and learnings

Q87. What was the hardest part of this project?

Answer:

The **join-request state machine** — getting every guard right: own-post 400, duplicate 409, non-owner 403, already-responded 409, and the accept path mutating the post while notifying.

It taught me that "happy path" code is easy; the value is in the **edge conditions and the status transitions**.

Follow-up you may get:

- How did you verify it?
- Any bug you hit here?

Q88. Tell me about a bug you found and fixed.

Answer:

CORS mismatch: my backend allowed origin `localhost:3000` while the Vite dev server runs on `5173` — cross-origin calls from the dev app would be rejected.

Diagnosis: browser console CORS error + curl works (curl has no origin) -> pointed straight at the security config, not the endpoint.

Lesson: **environment assumptions belong next to config**, and I now diff dev/prod origins before demos.

Follow-up you may get:

- How did you confirm it was not a token issue?
- Any frontend bug?

REMEMBER: This bug is REAL in the repo — if you use it, be ready to show SecurityConfig line 66.

Q89. What happens when a token expires while the user is using the app?

Answer:

The next API call returns **401**, `request()` throws `ApiError(401, ...)` , the screen shows the error.

Current gap: **no automatic logout/redirect** — I would add a global 401 interceptor: clear token, redirect to `/login?next=<page>`.

That is a UX bug I have already queued as an improvement.

Follow-up you may get:

- How would you implement the interceptor?
- Why no refresh token yet?

Q90. What would you do differently if you started over?

Answer:

Service layer + @Transactional from day one — multi-write flows deserve atomicity.

Schema in migrations (Flyway) instead of implicit `ddl-auto` — the DB would be reviewable in git.

And **pagination + unique email index immediately** — both are cheap up front, expensive to retrofit.

Follow-up you may get:

- What would you keep?
- Anything about the frontend?

Q91. How did you decide what to build first (prioritization)?

Answer:

Walking skeleton first: **routes + shell + health endpoint**, then the core loop (posts -> join -> accept), then nice-to-haves (notifications, recommendations, saved posts).

Rule: the **demo path must always work end-to-end** — polish on a screen only after its API flow is green.

Git history shows exactly that order — one feature area per commit.

Follow-up you may get:

- What did you cut?
- How did you handle scope creep?

Q92. Did any test fail that surprised you?

Answer:

The **duplicate join request case** needed a 409 — writing the test first made me realize my first implementation would have silently created a second row.

Adding the conflict guard in `submit()` was the fix; the test now locks that behavior (201 then 409).

Follow-up you may get:

- How do tests help design, not just verify?
- TDD or test-after?

Q93. How do you keep code quality up without a team reviewing you?

Answer:

Small, single-purpose commits with honest messages (13 clean commits, no "fix fix" noise).

Self-review checklist: every endpoint has an error path; every flow has a test; duplication noticed goes on a refactor list (the `currentUser()` duplication is item one).

And I re-read my own code a day later pretending to be the reviewer.

Follow-up you may get:

- What tools would you add? (linter/formatter — none in repo)
- How do you enforce style?

Q94. What is the most embarrassing weakness in this code?

Answer:

Probably: `PostController.list()` **loads the entire table and filters in Java** — it works for a demo, and it is exactly the code I would rewrite first with `Pageable`.

Pairing a real weakness with a precise fix turns it into a strength; pretending there is none is the failure mode.

Follow-up you may get:

- Why did you ship it that way?
- Show me the line you would change.

k) Managerial and behavioral questions (tied to this project)

Q95. How did you plan and manage this project?

Answer:

I broke it into **feature blocks** and committed each one green: scaffold -> tokens -> routing -> API client -> shell -> pages -> backend -> tests -> README.

I kept a simple rule: **never leave the demo path broken** — if a change endangered it, I finished or reverted before starting the next block.

The 13-commit history is the evidence of that discipline.

Follow-up you may get:

- How did you handle a block that grew too big?
- Any tool you used?

Q96. Tell me about a tough technical decision you made alone.

Answer:

H2 vs a real DB: I chose H2 for zero-setup demos, while designing so the swap to PostgreSQL is config-level (JPA abstraction, no native SQL).

Decision frame: what does this project optimize for (demoability + learning) vs what production needs (durability) — and I documented the trade-off rather than hiding it.

Follow-up you may get:

- What would the production cutover look like?
- Any decision you regret?

Q97. Suppose a teammate disagreed with your design (e.g., no service layer). How do you handle it?

Answer:

I would **ask for their concrete failure scenario** — "where does this break?" beats opinions.

Then compare honestly: my answer is "controllers are thin enough at this size, but you are right that transactions and reuse want a service" — if their case is stronger, I refactor; if not, I write the reasoning down so the next person sees it.

The artifact (code + README + tests) is the tie-breaker, not seniority.

Follow-up you may get:

- What if they are senior to you?
- How do you avoid conflict?

Q98. How did you handle deadlines?

Answer:

I delivered **end-to-end increments on a fixed cadence** — after each session the app ran and the demo path worked.

When scope threatened (nice-to-have screens), I cut by **risk-first order**: core flow > data display > polish.

That is why auth screens and recommendations came after the join-request loop was solid.

Follow-up you may get:

- What did you cut entirely?

- What if a deadline slipped?

Q99. What would you do differently as the tech lead on this?

Answer:

Put **guardrails before features**: migration files, CI running tests on every push, and error-shape tests — all cheap early, painful later.

Also define the **API contract first** (OpenAPI) so frontend and backend cannot drift; I did this informally with view models, but formalizing it scales to a team.

Follow-up you may get:

- How would you onboard a teammate?
- What about code review rules?

Q100. If a manager asked you to add a feature tomorrow, what would worry you?

Answer:

That the change touches **multi-write flows without @Transactional** (partial failure) and the **unpaginated feed** (performance regression).

My first hour: write a failing test for the new flow, then implement inside a transaction, then check the feed query plan/indexes.

I would also flag the CORS fix before any demo with the team.

Follow-up you may get:

- How do you estimate such work?
- How do you keep a release stable?

Q101. How do you prioritize quality vs speed?

Answer:

Quality on the money path (auth, join/accept, data integrity): tests + guards + correct status codes.

Speed on presentational work: shared shell and Tailwind tokens made screens fast without skipping review.

The tests I wrote are exactly on the money path — 3 tests, 3 highest-risk behaviors.

Follow-up you may get:

- What did you skip? (frontend tests, CI)
- Would that fly at scale?

Q102. Tell me about something you learned the hard way.

Answer:

That **implicit state is expensive**: the first version of duplicate join requests "just worked" until I thought about two clicks twice — then came the 409 guard and a test.

Generalizing it: every status field needs explicit transitions and a test per edge, or users WILL double-click.

Follow-up you may get:

- Any other lesson?
- How did it change your approach?

Q103. How do you explain technical choices to non-technical people?

Answer:

Analogy + impact: "JWT is like a wristband stamped at the entrance — staff verify it on the spot instead of checking a guest list every time."

Impact first: "This means faster responses and no shared session server; the trade-off is the wristband stays valid for 15 minutes."

Follow-up you may get:

- Explain CORS in one sentence.
- Explain why we need tests, to a PM.

Q104. Where do you see this project going?

Answer:

Near term: **production hardening** — PostgreSQL + Flyway, pagination, transactional writes, CI, the CORS/401 UX fixes.

Product: real password reset (screen exists, backend does not), email verification, team chat, real-time notifications.

Scale: caching the feed, async notifications, WebSocket delivery — mapped out in Section 11.

Follow-up you may get:

- What is the single next feature?
- How would you measure success?

I) Tricky / cross questions (and handling "I don't know")

Q105. You said "everything is secure" — then why is JWT in localStorage?

Answer:

I would not say "everything is secure" — I would say "**here is the boundary of what this app protects**": it prevents injection, hash-leakage, CSRF, and unauthorized access.

localStorage is a **deliberate simplicity trade-off**: immune to CSRF, readable by injected JS. The upgrade path is HttpOnly SameSite cookies with CSRF tokens, or in-memory + refresh tokens.

The honest framing wins more trust than a blanket claim.

Follow-up you may get:

- Which attack would you fix first?
- How do you detect XSS?

Q106. Your recommended-posts endpoint — is that machine learning?

Answer:

No — it is a **transparent heuristic**: count how many of my skills appear in a post's required-role skills (`PostController.score()`), sort descending.

It is explainable, free to compute, and a good fit for a small catalog.

At scale I would move to SQL-side scoring or a real ranking service — but I would never oversell it as ML.

Follow-up you may get:

- What happens if the user has no skills?
- How would you improve it?

REMEMBER: Never inflate. Calling a sort "ML" is how candidates lose all credibility in one sentence.

Q107. Why does GET /api/users/me require a token when SecurityConfig allows GET /api/users/**?

Answer:

Two layers: the URL rule says "no login needed to REACH the handler", but `UserController.me()` needs the **principal to know WHICH user** — with no token it throws 401 from `currentUserId()`.

So the endpoint is "publicly routed but identity-dependent" — a subtle distinction I would clean up by excluding `/me` from the `permitAll` matcher for clarity.

Follow-up you may get:

- Is that a security hole? (No — it fails closed)
- How would you restructure the rule?

REMEMBER: A cross question like this tests whether you actually READ your config. If unsure: "Let me trace both layers..." and reason aloud.

Q108. What does your app do if two users accept the same join request simultaneously?

Answer:

Both checks read `pending` before either writes — possible double-processing; **no locking exists in the code today**.

Mitigations in order: `@Version` optimistic lock on `JoinRequest` (second save fails -> 409), or a conditional UPDATE `SET status=accepted WHERE id=? AND status="pending"` affecting 0 rows -> 409.

That is exactly why transactions belong around `respond()`.

Follow-up you may get:

- Optimistic vs pessimistic?
- What HTTP code would the loser get?

Q109. If you don't know an answer — what do you say?

Answer:

Never bluff. Use the 3-step frame: (1) "I have not hit that scenario in this project"; (2) reason it out from what I DO know: "Here is how I would approach it — starting from the code I have"; (3) state the verification: "I would confirm with a test / the docs."

Example: "I have not configured JWT key rotation — my first step would be the JJWT docs plus a dual-key verification window, since `parseSignedClaims` is where validation happens."

Follow-up you may get:

- Can you try to answer anyway?
- What would you read first?

REMEMBER: Interviewers often ask something impossible just to watch you think. Reasoning aloud beats a wrong confident answer.

Q110. Isn't filtering in memory just wrong?

Answer:

For the **current scale** (hundreds of seeded/demo rows) it is correct and simple — and the client-side useMemo makes the UX instant.

It is **not a production design** — at 100k posts it is the first thing to die, and the fix (SQL WHERE + pagination + index) is on my improvement list.

So: "right for now, wrong forever" — I keep both halves of that sentence together.

Follow-up you may get:

- What is the exact threshold you would care about?
- Show me the refactor.

Q111. Why do you have BOTH server-side and client-side filters on the feed?

Answer:

Server params exist for **other consumers and cold loads**; the client useMemo gives **zero-latency filtering** for what is already loaded.

In a bigger system I would push everything server-side with pagination and keep the client filter only for the current page — right now the page loads the full list, so client filtering is honest.

Follow-up you may get:

- Which one wins for a fresh search?
- What changes with pagination?

Q112. Is this project really yours end to end? Explain any file.

Answer:

Yes — and I prove it by **picking any file cold**: e.g., `JwtAuthFilter` (why it never blocks the chain), `applyUpdate` (PATCH semantics), `useFetch` (the active flag), `SecurityConfig` (why CORS origin is wrong).

If I hit a file I barely remember, I read it aloud and reason — trained on my own code, not memorized scripts.

Follow-up you may get:

- Walk me through DataSeeder.
- What does index.css do?

REMEMBER: Before the interview, re-open 5 random files and explain each in 60 seconds. This question is common and fatal if you freeze.

m) Scenario questions ("what if..." under pressure)

Q113. What if 10,000 users hit GET /api/posts at once?

Answer:

Today: every hit runs `findAll()` with no cache/index -> DB CPU spikes and latency climbs fast.

Immediate: **cache the first page** (Caffeine/Redis, 30s TTL) + serve `Cache-Control`; most users only scroll page 1.

Structural: pagination + index on `(status, created_at desc)`; horizontal instances behind LB are easy because JWT is stateless.

Watch p95 and DB connections; scale reads with a replica if needed.

Follow-up you may get:

- What if they all hit page 50?
- Where do you add rate limiting?

Q114. What if the database goes down at 2 AM?

Answer:

Behavior now: every request 500s with the safe generic message; `handleGeneric` logs the cause — **no crash of the JVM**.

Ops response: readiness probe fails -> LB stops routing; alerts fire on 5xx spike.

Mitigations: connection pool timeouts so requests fail fast; circuit breaker on the data layer; for prod: HA DB (primary + replica + backups). H2 in-memory means today = restart + DataSeeder reseeds demo data.

Follow-up you may get:

- What is lost? (demo data only)
- How would you design for zero data loss?

Q115. What if a user creates a post with a 10MB description?

Answer:

No size limit is configured — the servlet container / server would accept it; DB row grows, feed payload bloats.

Fixes: **validation** (`@Size(max=5000)` on description), and at the edge a request-size limit (server `max-http-request-size` or gateway limit).

Frontend: cap textarea length with a counter for early feedback.

Follow-up you may get:

- Where exactly would you add `@Size`?
- What about file uploads? (Not supported at all)

Q116. What if someone brute-forces login passwords?

Answer:

Nothing in the repo slows them down — **no rate limiting** (a real limitation).

Plan: per-IP + per-account throttling (e.g., 5 attempts/min at the gateway or a filter), exponential backoff, account lockout with unlock flow, plus alerts on failure spikes.

BCrypt cost 12 already raises per-attempt CPU cost for the attacker.

Follow-up you may get:

- Where would you implement it?

- How do you avoid locking out real users?

Q117. What if the frontend and backend are deployed on different domains?

Answer:

CORS must allow the real origin — current config only allows `http://localhost:3000`, so I would set the deployed origin(s) in `corsConfigurationSource()` (or make it env-driven).

Then: same Origin header rules, credentials if cookie-based, preflight for PATCH (non-simple method) — my config already allows PATCH/OPTIONS.

With header-based JWT, no credentials flag is strictly needed.

Follow-up you may get:

- What does preflight do?
- Why is PATCH special?

Q118. What if I told you your API is returning wrong status codes?

Answer:

First: **reproduce** — curl the endpoint and look at the actual status + body; compare with my design table (401/403/404/409/400).

Then check `ApiExceptionHandler` mappings (e.g., is something swallowed by the catch-all 500?) and the SecurityConfig entry point (401 JSON).

Finally: MockMvc test asserting `status().isXxx()` so it cannot regress.

Follow-up you may get:

- Example: 500 where 404 expected — why?
- How fast can you fix?

Q119. What if you had to ship this with a junior teammate tomorrow?

Answer:

I would **hand over the contract first**: README (setup + endpoints + error shape), the 3 tests as living examples, and a walkthrough of the join-request state machine.

Then split by boundary: they take frontend screens against `api.js`; I take backend changes + reviews — boundaries prevent stepping on each other.

And I would add the missing CI check so "green" is not a matter of opinion.

Follow-up you may get:

- How do you review their PR?
- What could go wrong?

13. Cheat Sheet (one-page rapid revision)

Stack in one line

React 19 + Vite 7 + Tailwind 4 SPA --> fetch + Bearer JWT --> Spring Boot 3.3 (Java 22) controllers --> Spring Data JPA --> H2 in-memory; errors via `@RestControllerAdvice`; auth via `JwtAuthFilter`.

Key numbers (memorize)

Number	Meaning
14	Frontend routes (App.jsx)
7	Backend controllers
6	JPA entities (User, Post, PostRole, JoinRequest, Comment, Notification)
23	approx. API endpoints (auth 2, posts 6, comments 2, join/requests 3, me 5, users 4, health 1)
15 min	JWT expiry (900000 ms)
BCrypt 12	Password hashing strength
5 / 4	Seeded users / seeded posts (DataSeeder)
3	MockMvc integration tests
13	Git commits
4000 / 5173	Backend port / frontend dev port
teamup-demo-2026	Demo password for all seeded users
{status, message}	The one error JSON shape

Three flows to be able to draw from memory

```
LOGIN:    page -> POST /api/auth/login -> findByEmail + bcrypt matches -> JWT -> localStorage
          later calls -> Authorization: Bearer -> JwtAuthFilter -> SecurityContext -> controller

JOIN:     POST /api/posts/{id}/join -> guards(400 own post, 409 dup) -> save pending
          -> notify creator; PATCH ../requests/{id} {action} -> owner check(403) ->
          pending check(409) -> accept: addMember + save -> notify requester

FEED:     GET /api/posts -> findAll ordered desc -> (in-memory filters) -> [PostView]
          -> client useMemo filters instantly
```

Top 15 questions to prepare cold

- 1. Tell me about this project in 2 minutes.
- 2. Why Spring Boot + React instead of alternatives?
- 3. Walk me through login, end to end.
- 4. How does your JWT filter work? What if the token expires mid-session?
- 5. Where do you check that only the creator can edit a post? (PostController.update line ~121)
- 6. What does `@RestControllerAdvice` give you over try/catch in each method?

- 7. Explain your database schema and the join_requests state machine.
- 8. Why `@ElementCollection` for skills, but an entity for post_roles?
- 9. What is the N+1 problem and where could it appear in your code?
- 10. Your feed does `findAll()` then filters in memory — is that OK at scale? What would you change?
- 11. Where is authorization enforced — annotation or code? Both levels?
- 12. How does the frontend know who you are after a refresh?
- 13. What tests do you have? What do they NOT cover?
- 14. What is the weakest part of this system, and how would you fix it?
- 15. How would you handle 10,000 users hitting the feed API at once?

Top 10 things NEVER to say

- "I don't know how that works" about code you shipped — say instead: "Let me reason through it..." and walk the code path.
- "We used microservices" — this is a monolith; own it: "layered monolith, extracted services are a next step."
- "It is fully secure / production-ready" — no HTTPS, no rate limiting, CORS gap; be precise.
- "There is no pagination because SQL handles it" — there is genuinely no pagination; name it as a limitation.
- "I used Redis / Docker / Kubernetes" — none are in the repo. Describe them as future work.
- "My teammate did the backend" (if you built it solo) — inconsistency kills trust; know EVERY file.
- "It crashed and I don't know why" — always have a debugging story (logs, curl, MockMvc test).
- "JWT is stored in a cookie" (it is not — it is localStorage) — know exactly where your token lives.
- "H2 is a production database" — it is the demo choice; say PostgreSQL is the production path.
- "I would add more features" as the only improvement — lead with quality (transactions, indexes, tests).

INTERVIEW TIP: Final revision order: cheat sheet -> Section 5 flows -> Section 7 code -> Q&A category (j) challenges. That covers 80% of what managers actually ask.